Probabilistic matching: Fellegi–Sunter on what the rules left

For the records the deterministic stage left unresolved:

1. **Candidates:** generate pairs against all records: exact and compound keys, one-deletion name keys, and sorted neighbourhood on the name and on the reversed name. Every potential match shares a key, so none depends on a sorted window, whose reach shrinks as the file grows. `SPEC` also offers the phone alone and TF-IDF, which `FINAL` replaces because their work grows with N^2.
2. **Scoring:** Splink's **Fellegi–Sunter** model scores exactly those pairs. It is fitted on train: λ from near-certain rules, u by random sampling, m by EM. A score doesn't depend on the size of the scored set: term frequencies come from train, and λ is rescaled to the scored set's size.
3. **Decision:** accept at `match_weight ≥` a threshold.
4. **Resolution:** cluster with the deterministic pairs. A pair may also join two clusters the rules formed.

How configurations are chosen, without overfitting:

- **The threshold** comes from one rule on **out-of-fold** train scores (entity k-fold CV), for every configuration: `SPEC`, each decision variant, each Optuna trial.
- **Variants** are adopted only by a rule (`adopt`): a paired gain on cross-validated train and no loss on val. Optuna's suggestion is judged by the same rule.
- **test** is scored once.

The configuration flows `SPEC` (as written) → `TUNED` (threshold set) → `FINAL` (decisions applied); no cell reassigns them.

Cells: Configuration · Setup · Candidates (train) · Training · Threshold · A/B (val, paired) · Decisions · Will it scale (FINAL) · Evaluation (val) · Why a pair matches · Output (train / val / test → artifacts, gold) · Optuna + Pareto.

In [1]:
```
import json
import time
from dataclasses import replace

import optuna
import pandas as pd
```

```
from IPython.display import display

from er.candidates import candidates, estimate, found_by
from er.config import (Budget, Comparator, Comparison, DeletionKeys, FSModel, Key, KeyBlocker, Level, OverBudget,
ProbSpec, ProbTune, Resolver,
                       SortedNeighbourhood, TfidfNeighbours, ThresholdRule, with_level, without_tf)
from er.evaluate import adopt, candidate_metrics, missed_pairs, paired_test, predicted_pairs, true_pairs
from er.io import append_leaderboard, assert_test_scored_once, load, read_stage, test_evaluations, write_final, wr
ite_gold, write_stage
from er.match import parameters, train_fs, u_values, waterfall
from er.pipeline import (ProbResult, accept_curve, cross_validate_fs, cv_threshold_curve, entity_folds, run_fs, sc
ale_check,
                         select_threshold, summed, threshold_curve, window_only)
from er.settings import load_settings
from er.standardize import prepare
from er.strings import connect
from er.tune import new_study, pick_within_noise, plain, start_dashboard
```

Configuration

The only cell, with `config/settings.toml`, that names columns. Correlated columns share one comparison (address code / normalized address / postal form *address*), because Fellegi–Sunter assumes independence.

In [2]:
```
S = load_settings()
DATA = S.data
FOLD, COMPANY = ("kana_fold",), ("nfkc", "strip_legal_form", "kana_fold")

GENERATORS = (
    KeyBlocker(name="phone", keys=(Key("PHONE_NUM_4"),)),
    KeyBlocker(name="address", keys=(Key("ADDRESS", ("address",)),)),
    KeyBlocker(name="address_code", keys=(Key("SOURCE_ADDR_CD"),)),
    KeyBlocker(name="cif", keys=(Key("CIF_ID"),)),
    KeyBlocker(name="person_name", keys=(Key("PERSON_NAME", FOLD),)),
    KeyBlocker(name="company_name", keys=(Key("NYS_CNV_P_NAME_KANA", COMPANY),)),
    SortedNeighbourhood(name="sn_person_name", key=Key("PERSON_NAME", FOLD), window=4),
    SortedNeighbourhood(name="sn_company_name", key=Key("NYS_CNV_P_NAME_KANA", COMPANY), window=4),
    TfidfNeighbours(name="tfidf_person_name", key=Key("PERSON_NAME", FOLD), ngram=(2, 3), k=5, min_cosine=0.5),
    TfidfNeighbours(name="tfidf_company_name", key=Key("NYS_CNV_P_NAME_KANA", COMPANY), ngram=(2, 3), k=5, min_cos
```

```
ine=0.5),
)
# generators whose pairs grow about linearly with N, for the scale decisions: the phone alone (10,000 values) and
# TF-IDF (every query against every record) grow with N² ("Will it scale?" below). One-deletion keys find every na
me one
# edit away at any size; a sorted window reaches fewer of them as the file grows.
SCALABLE = (
    KeyBlocker(name="phone_prefecture", keys=(Key("PHONE_NUM_4"), Key("ADDRESS_ONE"))),
    KeyBlocker(name="phone_postal_area", keys=(Key("PHONE_NUM_4"), Key("POSTAL", ("first3",)))),
    KeyBlocker(name="phone_name_initial", keys=(Key("PHONE_NUM_4"), Key("ANY_NAME", ("initial",)))),
    SortedNeighbourhood(name="sn_person_name_reversed", key=Key("PERSON_NAME", ("kana_fold", "reverse")), window=
4),
    DeletionKeys(name="company_name_deletions", key=Key("NYS_CNV_P_NAME_KANA", COMPANY)),
    DeletionKeys(name="person_name_deletions_postal_area", key=Key("PERSON_NAME", FOLD), within=(Key("POSTAL", ("f
irst3",)),)),  # alone: N^1.7
    KeyBlocker(name="given_name_postal", keys=(Key("GIVEN_NAME_KANA", FOLD), Key("POSTAL"))),  # surname change
)
NAME_KEYS = ("person_name", "company_name", "company_name_deletions", "person_name_deletions_postal_area", "given_
name_postal")
BY_NAME = {b.name: b for b in GENERATORS + SCALABLE}

SPEC = ProbSpec(
    standardize=S.standardize,
    previous_stage="deterministic",
    stage="probabilistic",
    blockers=GENERATORS,
    model=FSModel(
        name="fs",
        comparisons=(
            Comparison(name="name", null_if_missing=("PERSON_NAME",), tf=Key("PERSON_NAME", FOLD), levels=(
                Level(label="exact", any_of=(("PERSON_NAME", Comparator("exact")),), tf=True),
                Level(label="fold exact", any_of=(("PERSON_NAME", Comparator("exact", normalize=FOLD)),), tf=Tru
e),
                Level(label="fold JW high", any_of=(("PERSON_NAME", Comparator("jaro_winkler", 0.92, FOLD)),)),
                Level(label="given name", any_of=(("GIVEN_NAME_KANA", Comparator("exact", normalize=FOLD)),)),  #
surname change
                Level(label="fold JW low", any_of=(("PERSON_NAME", Comparator("jaro_winkler", 0.85, FOLD)),)),
            )),
            Comparison(name="company", null_if_missing=("NYS_CNV_P_NAME_KANA",), tf=Key("NYS_CNV_P_NAME_KANA", COM
PANY), levels=(
                Level(label="exact", any_of=(("NYS_CNV_P_NAME_KANA", Comparator("exact", normalize=COMPANY)),), tf
```

```
=True),
                Level(label="JW high", any_of=(("NYS_CNV_P_NAME_KANA", Comparator("jaro_winkler", 0.92, COMPAN
Y")),)),
                Level(label="JW low", any_of=(("NYS_CNV_P_NAME_KANA", Comparator("jaro_winkler", 0.85, COMPAN
Y")),)),
            )),
            Comparison(name="address", null_if_missing=("SOURCE_ADDR_CD", "ADDRESS", "POSTAL"), levels=(
                Level(label="same address", any_of=(("SOURCE_ADDR_CD", Comparator("exact")),
                                                    ("ADDRESS", Comparator("exact", normalize=("address",)))))),
                Level(label="address close", any_of=(("ADDRESS", Comparator("levenshtein", 2, ("address",))),)),
                Level(label="same postal", any_of=(("POSTAL", Comparator("exact")),)),
            )),
            Comparison(name="phone", null_if_missing=("PHONE_NUM_4",), levels=(
                Level(label="exact", any_of=(("PHONE_NUM_4", Comparator("exact")),)),
                Level(label="one digit off", any_of=(("PHONE_NUM_4", Comparator("hamming", 1)),)),
            )),
            Comparison(name="cif", null_if_missing=("CIF_ID",), levels=(
                Level(label="exact", any_of=(("CIF_ID", Comparator("exact")),)),
            )),
        ),
        prior_rules=((Key("CIF_ID"), Key("FULL_NAME_KANA")), (Key("CIF_ID"), Key("NYS_CNV_P_NAME_KANA")),
                     (Key("FULL_NAME_KANA"), Key("SOURCE_ADDR_CD"), Key("PHONE_NUM_4"))),
        prior_recall=0.5,  # those rules recall about half of the true pairs (deterministic exact stage, measured)
        u_max_pairs=1e7,
        m_method="em",
        em_rules=((Key("PHONE_NUM_4"),), (Key("ADDRESS", ("address",)),)),
        seed=0,
    ),
    min_match_weight=4.0,  # starting value; THRESHOLD_RULE sets the real one (Threshold cell)
    resolver=Resolver(method="connected_components"),
    budget=Budget(max_candidate_pairs=5_000_000, max_block_size=5_000, target_rows=1_000_000, benchmark_pairs=20_0
00),
)

CV_FOLDS, CV_SEED = 3, 0
THRESHOLD_RULE = ThresholdRule(grid=(0, 2, 4, 6, 8, 10, 12, 14, 16), max_fp=0, max_fp_rate_hi=0.05, margin_steps=
1)



def optuna_suggestion(s: ProbSpec) -> ProbSpec:  # an Optuna study's suggestion (trial 12); passed adopt vs FINAL
```

```
    model = without_tf(s.model)
    for (comparison, label), threshold in {("name", "fold JW high"): 0.95, ("name", "fold JW low"): 0.75,
                                           ("company", "JW high"): 0.97, ("company", "JW low"): 0.80}.items():
        model = with_level(model, comparison, label, threshold=threshold)
    off = {"person_name", "company_name", "sn_person_name", "tfidf_company_name"}
    return replace(s, blockers=tuple(b for b in GENERATORS if b.name not in off), model=model)


CHANGES = {  # each variant, defined once as a change to a configuration
    "labels": lambda s: replace(s, model=replace(s.model, m_method="labels")),
    "no_term_frequency": lambda s: replace(s, model=without_tf(s.model)),
    "exact_keys_only": lambda s: replace(s, blockers=tuple(b for b in s.blockers if isinstance(b, KeyBlocker))),
    "levenshtein_names": lambda s: replace(s, model=with_level(with_level(s.model, "name", "fold JW high", functio
n="levenshtein",
                                                                          threshold=1), "name", "fold JW low", fun
ction="levenshtein", threshold=2)),
    "constrained_given_name": lambda s: replace(s, resolver=Resolver(method="constrained", cannot_link=("GIVEN_NAM
E_KANA",))),
    "optuna_suggestion": optuna_suggestion,
    "u_from_1e6_pairs": lambda s: replace(s, model=replace(s.model, u_max_pairs=1e6)),  # u-sampling ≈ 6× cheaper
    "compound_phone_keys": lambda s: replace(s, blockers=tuple(b for b in s.blockers if b.name != "phone") + tuple
(
        BY_NAME[n] for n in ("phone_prefecture", "phone_postal_area", "phone_name_initial"))),
    "sorted_names_for_tfidf": lambda s: replace(s, blockers=tuple(b for b in s.blockers if not isinstance(b, Tfidf
Neighbours)) + tuple(
        BY_NAME[n] for n in ("sn_person_name", "sn_person_name_reversed") if BY_NAME[n] not in s.blockers)),
    "name_keys": lambda s: replace(s, blockers=s.blockers + tuple(BY_NAME[n] for n in NAME_KEYS if BY_NAME[n] not
in s.blockers)),
    "bridge_rule_clusters": lambda s: replace(s, bridge=True),  # stage 2 may join two clusters the rules formed
}
# adopted in order, each on top of the choice so far: "better" needs a significant paired gain on cross-validated
train
# (p < DECISION_P, no FP added); "cheaper" and "safer" (more pairs, gain at scale: "Will it scale?") need no paire
d loss
# there. All need no paired loss on val (er.evaluate.adopt).
DECISIONS = (("no_term_frequency", "better"), ("exact_keys_only", "cheaper"), ("optuna_suggestion", "better"),
             ("u_from_1e6_pairs", "cheaper"), ("compound_phone_keys", "cheaper"), ("sorted_names_for_tfidf", "chea
per"),
             ("name_keys", "safer"), ("bridge_rule_clusters", "better"))
DECISION_P = 0.05
```

```
# the worked example ("Why a pair matches"): columns shown; the example match has these filled on both sides, the
first differing
EXPLAIN_SHOW = ("FULL_NAME_KANA", "NYS_CNV_P_NAME_KANA", "ADDRESS", "PHONE_NUM_4", "CIF_ID")
EXPLAIN_FILLED = ("FULL_NAME_KANA", "ADDRESS", "CIF_ID")

PTUNE = ProbTune(
    toggle_blockers=("person_name", "company_name", "sn_person_name", "sn_person_name_reversed", "sn_company_nam
e"),  # all scalable
    m_methods=("em", "labels"),
    tf_options=(True, False),
    level_thresholds={("name", "fold JW high"): (0.88, 0.97, 0.01), ("name", "fold JW low"): (0.75, 0.90, 0.01),
                      ("company", "JW high"): (0.88, 0.97, 0.01), ("company", "JW low"): (0.75, 0.90, 0.01)},
    resolvers={"connected_components": Resolver(method="connected_components"),
               "constrained_given_name": Resolver(method="constrained", cannot_link=("GIVEN_NAME_KANA",))},
    n_trials=30, seed=0,
    storage_path=S.paths["optuna"] / "probabilistic_matching.db", study_prefix="probabilistic",
    dashboard_host="127.0.0.1", dashboard_port=8081,  # 8080 belongs to the deterministic notebook
)
```

Setup

Three caches skip repeated work without changing any result:

- **Models,** by (config, fitting set).
- **u, per comparison,** by `FSModel.u_keys()`: a comparison's u depends only on itself and the fitting records, so every model that shares a comparison reuses its u (identical results; see tests/). Optuna trials mostly change name and company thresholds, so address and phone u are sampled once per fold.
- **Runs,** by (config, dataset, threshold).

In [3]:
```
CON, DB = connect()  # DuckDB with character-level cl_* string functions
uid = DATA.unique_id
l, r = f"{uid}_l", f"{uid}_r"
SHOW = ["stage", "TP", "FP", "recall", "recall_lo", "recall_hi", "fp_rate_hi", "pairs_FP", "overmerged", "dup_unre
solved"]
train_records, train_truth = load(DATA, "train")
val_records, val_truth = load(DATA, "val")
train_prev_pairs, train_prev_resolved = read_stage(DATA, "train", SPEC.previous_stage)
```

```
FOLDS = entity_folds(train_truth, DATA, CV_FOLDS, CV_SEED)
MODELS, U, RUNS, CVS = {}, {}, {}, {}


def fit(spec: ProbSpec, records: pd.DataFrame, truth: pd.DataFrame, key: object) -> dict:
    if (repr(spec.model), key) not in MODELS:
        df = prepare(spec.standardize, records, spec.normalized(), spec.used())
        u_keys = spec.model.u_keys()
        model_json = train_fs(spec.model, df, DB, uid, labels=truth.set_index(uid)[DATA.entity_id],
                              u={c: U[k, key] for c, k in u_keys.items() if (k, key) in U})
        for c, u in u_values(model_json).items():
            U.setdefault((u_keys[c], key), u)
        MODELS[repr(spec.model), key] = model_json
    return MODELS[repr(spec.model), key]


def run(spec: ProbSpec, dataset: str) -> ProbResult:  # model fitted on all of train
    key = (repr(spec), dataset)
    if key not in RUNS:
        records, truth = load(DATA, dataset)
        RUNS[key] = run_fs(spec, fit(spec, train_records, train_truth, "train"), records, truth,
                           *read_stage(DATA, dataset, spec.previous_stage), DB, DATA)
    return RUNS[key]


def cv(spec: ProbSpec) -> list[ProbResult]:  # out-of-fold results on train
    if repr(spec) not in CVS:
        CVS[repr(spec)] = cross_validate_fs(spec, fit, train_records, train_truth, train_prev_pairs, train_prev_re
solved, FOLDS, DB, DATA)
    return CVS[repr(spec)]


def threshold_of(spec: ProbSpec) -> float:  # THRESHOLD_RULE on this configuration's out-of-fold scores (no resolu
tion needed)
    return select_threshold(accept_curve(cv(spec), list(THRESHOLD_RULE.grid)), THRESHOLD_RULE)[0]


def joined(results: ProbResult | list[ProbResult]) -> set[tuple[str, str]]:  # every pair the clusters put togethe
r
    return set().union(*(predicted_pairs(x.records.set_index(uid)["CLUSTER_ID"]) for x in (results if isinstance(r
esults, list) else [results])))
```

```
def truth_set(truth: pd.DataFrame) -> set[tuple[str, str]]:
    return set(map(tuple, true_pairs(truth.set_index(uid)[DATA.entity_id], uid).to_numpy()))
```

Candidates on train: what each generator reaches, and what it costs

Every generator, `SPEC`'s and the scalable ones, on train only, so the design never sees val. Per generator: its work now, the share of the true pairs the deterministic stage missed that it finds (alone, then cumulatively in this order), the different-entity pairs it also makes, and its pairs projected to `budget.target_rows` (`scale_check`, α from entity subsamples). No scoring.

In [4]:
```
ALL = replace(SPEC, blockers=GENERATORS + SCALABLE)
t_df = prepare(ALL.standardize, train_records, ALL.normalized(), ALL.used())
t_entity = train_truth.set_index(uid)[DATA.entity_id]
t_prev = train_prev_resolved.set_index(uid)["cluster_id"].reindex(t_df[uid].to_numpy())
targets = missed_pairs(t_prev, t_entity.reindex(t_prev.index), uid)
cands, report = candidates(ALL.blockers, t_df, uid, set(t_prev.index[t_prev.map(t_prev.value_counts()).eq(1).to_nu
mpy()]), ALL.budget)


def reach(names: set[str]) -> dict[str, float]:  # the candidates of these generators, against the deterministic s
tage's misses
    return candidate_metrics(cands[found_by(cands, names)], t_entity, uid, targets)


alone = pd.DataFrame([reach({b.name}) for b in ALL.blockers])
GROWTH = scale_check(ALL, train_records, train_truth, DATA).set_index("key")  # per generator; reused by "Will it
scale?" and Optuna
report = report.assign(finds=alone["completeness_on_targets"], cumulative=[reach({b.name for b in ALL.blockers[: i
+ 1]})["completeness_on_targets"]
                                                                            for i in range(len(ALL.blockers))],
                       different_entity_pairs=alone["candidates"] - alone["true_in_candidates"]).join(GROWTH[["alp
ha", "projected_pairs"]], on="generator")
print(f"train: {len(targets):,} true pairs missed by the deterministic stage")
display(report[["generator", "big_o", "filled", "pairs", "seconds", "finds", "cumulative", "different_entity_pair
s", "alpha", "projected_pairs"]].round(4))
```

```
train: 2,135 true pairs missed by the deterministic stage
```

	generator	big_o	filled	pairs	seconds	finds	cumulative	different_entity_pairs	alpha	projected_pairs
0	phone	O(N + P)	O(30,147 + 50,676)	43054	0.069	0.5035	0.5035	41979	1.80	27296242
1	address	O(N + P)	O(30,147 + 15,675)	11534	0.060	0.3770	0.7607	10729	1.58	4024827
2	address_code	O(N + P)	O(30,147 + 14,581)	10689	0.056	0.3429	0.8061	9957	1.57	3600997
3	cif	O(N + P)	O(30,147 + 3,136)	230	0.056	0.1077	0.8478	0	0.98	95670
4	person_name	O(N + P)	O(30,147 + 4,518)	1163	0.053	0.3831	0.9096	345	1.05	178732
5	company_name	O(N + P)	O(30,147 + 946)	547	0.046	0.0806	0.9349	375	1.31	93751
6	sn_person_name	O(N log N + N·w)	O(23,848·log 23,848 + 23,848·4)	63410	0.119	0.4089	0.9382	62537	1.00	3000000
7	sn_company_name	O(N log N + N·w)	O(5,009·log 5,009 + 5,009·4)	14243	0.091	0.0820	0.9386	14068	1.00	3000000
8	tfidf_person_name	O(N·L + U·N·q)	O(23,848·L + 21,967·23,848·14)	22001	2.486	0.4337	0.9433	21075	2.00	1000000000000
9	tfidf_company_name	O(N·L + U·N·q)	O(5,009·L + 21,967·5,009·18)	2416	0.269	0.0829	0.9443	2239	2.00	1000000000000
10	phone_prefecture	O(N + P)	O(30,147 + 3,208)	1211	0.052	0.1864	0.9443	813	1.17	193023
11	phone_postal_area	O(N + P)	O(30,147 + 3,840)	633	0.055	0.2674	0.9443	62	1.02	136184
12	phone_name_initial	O(N + P)	O(30,147 + 5,744)	2134	0.056	0.2642	0.9443	1570	1.20	389710
13	sn_person_name_reversed	O(N log N + N·w)	O(23,848·log 23,848 + 23,848·4)	63359	0.117	0.4782	0.9494	62338	1.00	3000000
14	company_name_deletions	O(N·L + P)	O(30,147·L + 1,501)	1082	0.069	0.0834	0.9494	904	1.44	234922
15	person_name_deletions_postal_area	O(N·L + P)	O(30,147·L + 3,551)	565	0.120	0.2571	0.9499	16	1.01	120382
16	given_name_postal	O(N + P)	O(30,147 + 2,939)	519	0.053	0.2365	0.9513	14	0.99	94905

Training on train: EM vs labels

The weight of a level is log2(m/u): how much more often true matches land there than random pairs. EM is unsupervised (usable on real data); labels is the supervised reference. A level without an estimate never uses Splink's defaults: no sampled pair → u at its 95% upper bound; never seen in training → m = u (0 bits). Those levels are listed.

In [5]:
```
n, k = len(t_df), len(SPEC.model.comparisons)
em_p = sum(estimate(KeyBlocker(name="em", keys=rule), t_df, n)["est_pairs"] for rule in SPEC.model.em_rules)
print(f"cost: u O(max_pairs·C) = O({SPEC.model.u_max_pairs:,.0f}·{k}) · EM O(iterations·P·C), P = {em_p:,} · label
```

```
s O(Σ k² per entity)")
DATA.models_dir.mkdir(parents=True, exist_ok=True)
weights = {}
for method in ("em", "labels"):
    spec, t = replace(SPEC, model=replace(SPEC.model, m_method=method)), time.perf_counter()
    model_json = fit(spec, train_records, train_truth, "train")
    (DATA.models_dir / f"{spec.model.name}_{method}.json").write_text(json.dumps(model_json, ensure_ascii=False, i
ndent=1))
    weights[method] = parameters(model_json).set_index(["comparison", "level"])[["m", "u", "weight"]]
    filled = model_json["er"]["filled"]
    print(f"{method}: fitted in {time.perf_counter() - t:.1f}s · no estimate: u {filled['u'] or '-'} · m {filled
['m'] or '-'}")
display(pd.concat(weights, axis=1).round(3))
```

```
cost: u O(max_pairs·C) = O(10,000,000·5) · EM O(iterations·P·C), P = 66,351 · labels O(Σ k² per entity)
em: fitted in 9.1s · no estimate: u - · m -
labels: fitted in 0.2s · no estimate: u - · m -
```

		em			labels		
		m	u	weight	m	u	weight
comparison	level						
name	exact	0.543	0.000	15.569	0.542	0.000	15.567
	fold exact	0.218	0.000	15.532	0.218	0.000	15.530
	fold JW high	0.090	0.000	11.374	0.090	0.000	11.378
	given name	0.110	0.000	7.811	0.108	0.000	7.790
	fold JW low	0.004	0.000	3.578	0.004	0.000	3.779
	else	0.035	0.999	-4.816	0.037	0.999	-4.741
company	exact	0.654	0.000	13.126	0.593	0.000	12.984
	JW high	0.012	0.000	9.511	0.010	0.000	9.194
	JW low	0.003	0.000	4.452	0.003	0.000	4.439
	else	0.331	1.000	-1.596	0.395	1.000	-1.341
address	same address	0.708	0.000	14.015	0.708	0.000	14.014
	address close	0.004	0.000	8.893	0.003	0.000	8.595
	same postal	0.113	0.000	10.823	0.114	0.000	10.839
	else	0.175	1.000	-2.515	0.175	1.000	-2.515
phone	exact	0.746	0.000	12.540	0.745	0.000	12.539
	one digit off	0.081	0.004	4.497	0.081	0.004	4.496
	else	0.173	0.996	-2.525	0.174	0.996	-2.518
cif	exact	0.464	0.000	15.920	0.472	0.000	15.942
	else	0.536	1.000	-0.901	0.528	1.000	-0.920

Threshold: chosen on cross-validated train

`THRESHOLD_RULE`: on out-of-fold train scores, the lowest weight with no false accepted pair and a stage FP upper bound ≤ `max_fp_rate_hi`, then `margin_steps` stricter. That gives `TUNED`.

In [6]:
```
t = time.perf_counter()
weight, curve = select_threshold(cv_threshold_curve(cv(SPEC), SPEC, train_prev_pairs, train_truth, FOLDS, DB, DAT
A,
                                                    list(THRESHOLD_RULE.grid)), THRESHOLD_RULE)
assert weight == threshold_of(SPEC)  # the full curve and the pair counts the rule reads pick the same weight
TUNED = replace(SPEC, min_match_weight=weight)
print(f"{CV_FOLDS}-fold CV: O(k · (fit + candidates + score + resolve)) in {time.perf_counter() - t:.1f}s → min_ma
tch_weight = {weight}")
display(curve[["min_match_weight", "accepted", "pairs_FP", "stage_fp_rate_hi", "TP", "FP", "recall", "recall_lo",
"recall_hi", "eligible"]].round(4))
```

```
3-fold CV: O(k · (fit + candidates + score + resolve)) in 32.3s → min_match_weight = 8.0
```

	min_match_weight	accepted	pairs_FP	stage_fp_rate_hi	TP	FP	recall	recall_lo	recall_hi	eligible
0	0	1654	15	0.0139	6494	18	0.9360	0.9300	0.9415	False
1	2	1607	14	0.0136	6448	17	0.9294	0.9231	0.9352	False
2	4	1503	1	0.0032	6360	1	0.9167	0.9100	0.9230	False
3	6	1397	0	0.0021	6260	0	0.9023	0.8951	0.9090	True
4	8	1216	0	0.0025	6080	0	0.8763	0.8684	0.8839	True
5	10	907	0	0.0033	5765	0	0.8309	0.8219	0.8396	True
6	12	488	0	0.0061	5326	0	0.7677	0.7576	0.7774	True
7	14	451	0	0.0066	5284	0	0.7616	0.7514	0.7715	True
8	16	386	0	0.0077	5220	0	0.7524	0.7421	0.7624	True

A/B on val, paired against TUNED

Each `CHANGES` entry applied to `TUNED`, compared on the same records: true pairs gained / lost, an exact sign test (`p`), false pairs added / removed.

In [7]:
```
val_true, base, ab = truth_set(val_truth), joined(run(TUNED, "val")), []
for name, change in CHANGES.items():
```

```
    spec = change(TUNED)
    m = run(spec, "val").metrics.iloc[-1]
    row = dict(recall=m["recall"], FP=m["FP"], fp_rate_hi=m["fp_rate_hi"], **paired_test(base, joined(run(spec, "v
al")), val_true),
               candidates=run(spec, "val").stage_metrics["candidates"])
    append_leaderboard(DATA, f"probabilistic:{name}", spec, "val", row)
    ab.append(dict(variant=name, **row))
display(pd.DataFrame(ab).round(4))
```

	variant	recall	FP	fp_rate_hi	TP_gains	TP_losses	p	FP_added	FP_removed	candidates
0	labels	0.8864	0	0.0015	0	0	1.0000	0	0	33871
1	no_term_frequency	0.8873	0	0.0015	2	0	0.5000	0	0	33871
2	exact_keys_only	0.8864	0	0.0015	0	0	1.0000	0	0	6724
3	levenshtein_names	0.8895	0	0.0015	8	1	0.0391	0	0	33871
4	constrained_given_name	0.7705	0	0.0017	0	258	0.0000	0	0	33871
5	optuna_suggestion	0.8936	0	0.0015	19	3	0.0009	0	0	14180
6	u_from_1e6_pairs	0.8806	0	0.0015	3	16	0.0044	0	0	33871
7	compound_phone_keys	0.8864	0	0.0015	0	0	1.0000	0	0	29441
8	sorted_names_for_tfidf	0.8864	0	0.0015	0	0	1.0000	0	0	52530
9	name_keys	0.8864	0	0.0015	0	0	1.0000	0	0	33902
10	bridge_rule_clusters	0.8882	0	0.0015	4	0	0.1250	0	0	36293

Decisions → FINAL

Each `DECISIONS` entry is applied on top of the choice so far, at its own `THRESHOLD_RULE` threshold, and adopted only if it passes `adopt`: paired on cross-validated train, plus no paired loss on val.

In [8]:
```
train_true, FINAL, verdicts = truth_set(train_truth), TUNED, []
for name, kind in DECISIONS:
    variant = CHANGES[name](FINAL)
    variant = replace(variant, min_match_weight=threshold_of(variant))
    on_cv = paired_test(joined(cv(FINAL)), joined(cv(variant)), train_true)
    on_val = paired_test(joined(run(FINAL, "val")), joined(run(variant, "val")), val_true)
    adopted = adopt(kind, on_cv, on_val, DECISION_P)
```

```
    verdicts.append(dict(change=name, rule=kind, adopted=adopted, **{f"cv_{x}": v for x, v in on_cv.items()},
                         val_TP_gains=on_val["TP_gains"], val_TP_losses=on_val["TP_losses"], val_FP_added=on_val
["FP_added"]))
    if adopted:
        FINAL = variant
display(pd.DataFrame(verdicts).round(4))
ADOPTED = [v["change"] for v in verdicts if v["adopted"]]
print(f"FINAL: {len(FINAL.blockers)} generators · term frequency {'on' if any(c.tf for c in FINAL.model.comparison
s) else 'off'} · "
      f"m by {FINAL.model.m_method} · min_match_weight {FINAL.min_match_weight} · {FINAL.resolver.method}")
```

	change	rule	adopted	cv_TP_gains	cv_TP_losses	cv_p	cv_FP_added	cv_FP_removed	val_TP_gains	val_TP_losses	val_FP_added
0	no_term_frequency	better	True	15	0	0.0001	0	0	2	0	0
1	exact_keys_only	cheaper	False	0	4	0.1250	0	0	0	0	0
2	optuna_suggestion	better	True	198	0	0.0000	0	0	62	1	0
3	u_from_1e6_pairs	cheaper	False	10	24	0.0243	0	0	0	3	0
4	compound_phone_keys	cheaper	True	0	0	1.0000	0	0	0	0	0
5	sorted_names_for_tfidf	cheaper	True	2	0	0.5000	0	0	0	0	0
6	name_keys	safer	True	1	0	1.0000	0	0	1	0	0
7	bridge_rule_clusters	better	True	12	0	0.0005	0	0	4	0	0

```
FINAL: 14 generators · term frequency off · m by em · min_match_weight 6.0 · connected_components
```

Will it scale, and does every potential match share a key?

`FINAL`'s generators, with their pairs projected to `budget.target_rows` (from the Candidates cell). Then the **potential matches** on train: true pairs whose fields would reach `min_match_weight` at that size (λ moved to it), and how many of them share no equality key (`window_only`). Only a sorted window could find those, and a window's reach shrinks as the file grows. The run stops here if either check fails. Training and tuning stay on train-sized data whatever the production size: λ is rescaled to the scored set, so a model fitted here scores a larger file correctly.

In [9]:
```
plan = GROWTH.loc[[b.name for b in FINAL.blockers]].reset_index()
display(plan)
assert plan["within_budget"].all(), f"over budget at {FINAL.budget.target_rows:,} rows: {plan.loc[~plan['within_bu
dget'], 'key'].tolist()}"
```

```
cover = window_only(FINAL, fit(FINAL, train_records, train_truth, "train"), train_records, train_truth, DB, DATA,
                    [len(train_records), FINAL.budget.target_rows])
display(cover)
assert cover["window_only"].eq(0).all(), "a potential match is left to a sorted window, whose reach shrinks as the file grows"
```

	key	columns	alpha	projected_pairs	within_budget
0	address	ADDRESS__address	1.58	4024827	True
1	address_code	SOURCE_ADDR_CD	1.57	3600997	True
2	cif	CIF_ID	0.98	95670	True
3	sn_company_name	NYS_CNV_P_NAME_KANA__nfkc_strip_legal_form_kan...	1.00	3000000	True
4	phone_prefecture	PHONE_NUM_4 + ADDRESS_ONE	1.17	193023	True
5	phone_postal_area	PHONE_NUM_4 + POSTAL__first3	1.02	136184	True
6	phone_name_initial	PHONE_NUM_4 + ANY_NAME__initial	1.20	389710	True
7	sn_person_name	PERSON_NAME__kana_fold	1.00	3000000	True
8	sn_person_name_reversed	PERSON_NAME__kana_fold_reverse	1.00	3000000	True
9	person_name	PERSON_NAME__kana_fold	1.05	178732	True
10	company_name	NYS_CNV_P_NAME_KANA__nfkc_strip_legal_form_kan...	1.31	93751	True
11	company_name_deletions	NYS_CNV_P_NAME_KANA__nfkc_strip_legal_form_kan...	1.44	234922	True
12	person_name_deletions_postal_area	PERSON_NAME__kana_fold + POSTAL__first3	1.01	120382	True
13	given_name_postal	GIVEN_NAME_KANA__kana_fold + POSTAL	0.99	94905	True

	records	potential	window_only
0	30147	6120	0
1	1000000	5034	0

Evaluation on val (FINAL)

`FINAL`'s candidates per generator: `completeness_on_targets` is the share of the deterministic stage's missed true pairs on the list, the ceiling for this stage. Then cumulative metrics per stage: the `deterministic` row reproduces the previous stage. `pairs_*` are matching metrics; the rest are resolution metrics. The transfer line checks the threshold, chosen on cross-validated train, against val: val's false-pair rate should stay within the CV upper bound. Then the false positives, and the true pairs just below the threshold.

In [10]:

```
res = run(FINAL, "val")
display(res.report[["generator", "big_o", "filled", "pairs", "completeness_on_targets", "seconds"]].round(4))
print(f"candidates {res.stage_metrics['candidates']:,} · they contain {res.stage_metrics['completeness_on_target
s']:.1%} of the true pairs the rules missed")
display(res.metrics[SHOW].round(4))
val_curve = threshold_curve(res, FINAL, read_stage(DATA, "val", FINAL.previous_stage)[0], val_truth, DB, DATA, lis
t(THRESHOLD_RULE.grid))
display(val_curve[["min_match_weight", "accepted", "pairs_FP", "TP", "FP", "recall", "fp_rate_hi"]].round(4))
cv_at = cv_threshold_curve(cv(FINAL), FINAL, train_prev_pairs, train_truth, FOLDS, DB, DATA, [FINAL.min_match_weig
ht]).iloc[0]
val_at = val_curve.set_index("min_match_weight").loc[FINAL.min_match_weight]
val_rate = val_at["pairs_FP"] / max(val_at["accepted"], 1)
print(f"transfer at weight {FINAL.min_match_weight}: CV {cv_at['pairs_FP']:.0f} false of {cv_at['accepted']:.0f} a
ccepted pairs "
      f"(rate ≤ {cv_at['stage_fp_rate_hi']:.2%}) · val {val_at['pairs_FP']:.0f} of {val_at['accepted']:.0f} ({val_
rate:.2%}) → "
      f"{'consistent' if val_rate <= cv_at['stage_fp_rate_hi'] else 'VAL ABOVE THE CV BOUND'}")
assert val_rate <= cv_at["stage_fp_rate_hi"], "the threshold does not transfer from cross-validated train to val"

cols = sorted(c for c in FINAL.used() if c in res.records)
side = res.records.set_index(uid)[cols]


def side_by_side(pairs: pd.DataFrame) -> pd.DataFrame:
    return pd.concat([pairs[["match_weight", "sources"]].reset_index(drop=True), side.loc[pairs[l]].add_suffix("_
l").reset_index(drop=True),
                      side.loc[pairs[r]].add_suffix("_r").reset_index(drop=True)], axis=1).T


accepted = res.scored["match_weight"] >= FINAL.min_match_weight
print(f"false positives accepted: {int((accepted & res.scored['fp']).sum())}")
if (accepted & res.scored["fp"]).any():
    display(side_by_side(res.scored[accepted & res.scored["fp"]].nlargest(DATA.show_rows, "match_weight")))
print("true pairs just below the threshold:")
display(side_by_side(res.scored[~accepted & ~res.scored["fp"]].nlargest(DATA.show_rows, "match_weight")))
```

	generator	big_o	filled	pairs	completeness_on_targets	seconds
0	address	O(N + P)	O(10,002 + 2,540)	1498	0.3690	0.010
1	address_code	O(N + P)	O(10,002 + 2,350)	1380	0.3133	0.008
2	cif	O(N + P)	O(10,002 + 1,041)	81	0.1186	0.008
3	sn_company_name	O(N log N + N·w)	O(1,640·log 1,640 + 1,640·4)	4810	0.0966	0.005
4	phone_prefecture	O(N + P)	O(10,002 + 819)	196	0.1772	0.007
5	phone_postal_area	O(N + P)	O(10,002 + 1,181)	167	0.2401	0.008
6	phone_name_initial	O(N + P)	O(10,002 + 1,468)	371	0.2650	0.008
7	sn_person_name	O(N log N + N·w)	O(7,909·log 7,909 + 7,909·4)	22577	0.4290	0.015
8	sn_person_name_reversed	O(N log N + N·w)	O(7,909·log 7,909 + 7,909·4)	22497	0.5256	0.015
9	person_name	O(N + P)	O(10,002 + 1,388)	320	0.3982	0.007
10	company_name	O(N + P)	O(10,002 + 217)	116	0.0966	0.004
11	company_name_deletions	O(N·L + P)	O(10,002·L + 273)	169	0.1010	0.012
12	person_name_deletions_postal_area	O(N·L + P)	O(10,002·L + 1,158)	194	0.2840	0.033
13	given_name_postal	O(N + P)	O(10,002 + 941)	181	0.2621	0.006

candidates 51,484 · they contain 92.5% of the true pairs the rules missed

	stage	TP	FP	recall	recall_lo	recall_hi	fp_rate_hi	pairs_FP	overmerged	dup_unresolved
0	deterministic	1544	0	0.6933	0.6738	0.7121	0.0019	0	0	1091
1	probabilistic	2042	0	0.9169	0.9047	0.9277	0.0015	0	0	314

	min_match_weight	accepted	pairs_FP	TP	FP	recall	fp_rate_hi
0	0	537	5	2091	6	0.9389	0.0056
1	2	523	1	2084	1	0.9358	0.0023
2	4	494	0	2063	0	0.9264	0.0015
3	6	474	0	2042	0	0.9169	0.0015
4	8	425	0	1994	0	0.8954	0.0015
5	10	300	0	1872	0	0.8406	0.0016
6	12	174	0	1730	0	0.7768	0.0017
7	14	136	0	1692	0	0.7598	0.0018
8	16	112	0	1669	0	0.7494	0.0018

transfer at weight 6.0: CV 0 false of 1443 accepted pairs (rate ≤ 0.21%) · val 0 of 474 (0.00%) → consistent
false positives accepted: 0
true pairs just below the threshold:

	0	1	2	3	4	5	6	7
match_weight	5.885354	5.885354	5.885354	5.885354	5.570724	5.567918	5.567918	5.567918
sources	phone_prefecture+phone_postal_area	phone_postal_area	phone_postal_area	phone_postal_area	sn_company_name+company_name+company_name_dele...	address+address_code+phone_prefecture+phone_po...	address+address_code+phone_prefecture+phone_po...	address+address_code+phone_prefecture+phone_po...
ADDRESS_l	<NA>	岐阜県高山市八軒町3-30-18	<NA>	<NA>	大阪府大阪市生野区勝山南1-18-10	神奈川県川崎市高津区東野川1-30-12	福岡県春日市上白水4-12-28	神奈川県横浜市南区六ツ川1-15-15
ADDRESS_ONE_l	茨城県	岐阜県	<NA>	<NA>	大阪府	神奈川県	福岡県	神奈川県
ADDRESS__address_l	<NA>	高山市八軒町3-30-18	<NA>	<NA>	大阪市生野区勝山南1-18-10	川崎市高津区東野川1-30-12	春日市上白水4-12-28	横浜市南区六ツ川1-15-15
ANY_NAME__initial_l	ソ	シ	マ	ア	カ	ニ	イ	オ
CIF_ID_l	9394783785889	5838503911029	2081339529034	2096382436427	1734023849310	8942740611332	3875582213575	1046260516723
FULL_NAME_KANA_l	<NA>	<NA>	<NA>	<NA>	<NA>	ニシカワハルミ	イノウエハル	オオモトトシヒコ
GIVEN_NAME_KANA__kana_fold_l	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	ハル	トシヒコ
NYS_CNV_P_NAME_KANA_l	カブシキガイシャゾディアッグキゴウテレビセイサクカイシャキゴウ	ジョウホウギジュツカイハツカブシキガイシャ	カブシキガイシャマジカルムーブ	アキタステーションビルカブシキガイシャ	カブシキガイシャカネコカイソウテン	<NA>	<NA>	<NA>
NYS_CNV_P_NAME_KANA__nfkc_strip_legal_form_kana_fold_l	ソテイアツクキコウテレヒセイサクカイシヤキコウ	シヨウホウキシユツカイハツ	マシカルムウフ	アキタステウシヨンヒル	カネコカイソウテン	<NA>	<NA>	<NA>
PERSON_NAME_l	<NA>	<NA>	<NA>	<NA>	<NA>	ニシカワハルミ	イノウエハル	オオモトトシヒコ
PERSON_NAME__kana_fold_l	<NA>	<NA>	<NA>	<NA>	<NA>	ニシカワハルミ	イノウエハル	オオモトトシヒコ
PERSON_NAME__kana_fold_reverse_l	<NA>	<NA>	<NA>	<NA>	<NA>	ミルハワカシニ	ルハエウノイ	コヒシトトモオ
PHONE_NUM_4_l	9687	7503	1854	6636	7842	9074	3064	4974
POSTAL_l	3170062	5060012	3580012	0292204	5440021	2130029	8160844	2320066
POSTAL__first3_l	317	506	358	029	544	213	816	232
SOURCE_ADDR_CD_l	<NA>	9695438217	<NA>	3393592139	2532423942	5533101708	5683835425	9152145076
ADDRESS_r	茨城県日立市平和町1-18-29	<NA>	埼玉県入間市東藤沢3-5-20	岩手県陸前高田市気仙町29-19	<NA>	神奈川県川崎市高津区東野川1-30-12	福岡県春日市上白水4-12-28	横浜市南区六ツ川１－１５－１５
ADDRESS_ONE_r	茨城県	<NA>	<NA>	岩手県	<NA>	神奈川県	福岡県	神奈川県
ADDRESS__address_r	日立市平和町1-18-29	<NA>	入間市東藤沢3-5-20	陸前高田市気仙町29-19	<NA>	川崎市高津区東野川1-30-12	春日市上白水4-12-28	横浜市南区六ツ川1-15-15
ANY_NAME__initial_r	ニ	チ	フ	ケ	カ	イ	タ	ア
CIF_ID_r	8513027962782	8676088365959	6827611087602	4433203496731	2262552829629	9883827822612	6215252974995	3927617677176
FULL_NAME_KANA_r	<NA>	<NA>	<NA>	<NA>	<NA>	イナバハルミ	タニグチハルナ	アサトシヒコ
GIVEN_NAME_KANA__kana_fold_r	<NA>	<NA>	<NA>	<NA>	<NA>	ハルミ	ハルナ	<NA>
NYS_CNV_P_NAME_KANA_r	ニッシンカサイカイジョウホケンカブシキガイシャ	チイキシンブンシャカブシキガイシャ	カブシキガイシャフコクセイメイホケン	ケイオウフェアウェルサポートカブシキガイシャ	カブシキガイシャカネコカイソウテン	<NA>	<NA>	<NA>
NYS_CNV_P_NAME_KANA__nfkc_strip_legal_form_kana_fold_r	ニツシンカサイカイシヨウホケン	チイキシンフンシヤ	フコクセイメイホケン	ケイオウフエアウエルサホウト	カネコカイソウテン	<NA>	<NA>	<NA>

	0	1	2	3	4	5	6	7
PERSON_NAME_r	<NA>	<NA>	<NA>	<NA>	<NA>	イナバハルミ	タニグチハルナ	アサトシヒコ
PERSON_NAME__kana_fold_r	<NA>	<NA>	<NA>	<NA>	<NA>	イナハハルミ	タニクチハルナ	アサトシヒコ
PERSON_NAME__kana_fold_reverse_r	<NA>	<NA>	<NA>	<NA>	<NA>	ミルハハナイ	ナルハチクニタ	コヒシトサア
PHONE_NUM_4_r	9687	7503	1854	6636	8829	9074	3064	4974
POSTAL_r	3170062	5060012	3580012	0292204	5440021	2130029	8160844	2320066
POSTAL__first3_r	317	506	358	029	544	213	816	232
SOURCE_ADDR_CD_r	1923269550	<NA>	5391676869	<NA>	<NA>	5533101708	5683835425	9152145076

Why a pair matches: evidence in bits

The prior (how likely two random records are the same entity) starts the score; each comparison then adds the bits log2(m/u) of the level the pair lands in, or 0 when a side is missing. Shown for the accepted true match with a typo closest to the threshold, and for the rejected non-match closest to it.

In [11]:
```
model = fit(FINAL, train_records, train_truth, "train")
shown = res.records.set_index(uid)[list(EXPLAIN_SHOW)]
left, right = shown.loc[res.scored[l]].set_index(res.scored.index), shown.loc[res.scored[r]].set_index(res.scored.
index)
filled = (left[list(EXPLAIN_FILLED)].notna() & right[list(EXPLAIN_FILLED)].notna()).all(axis=1)
differs = left[EXPLAIN_FILLED[0]].ne(right[EXPLAIN_FILLED[0]]).fillna(False)
examples = {"accepted true match with a typo": res.scored[accepted & ~res.scored["fp"] & filled & differs].nsmalle
st(1, "match_weight"),
            "rejected non-match closest to the threshold": res.scored[~accepted & res.scored["fp"]].nlargest(1, "m
atch_weight")}
for title, pair in examples.items():
    row = pair.iloc[0]
    print(f"{title}: {row['match_weight']:.2f} bits → match probability {row['match_probability']:.4f} · found by
{row['sources']}")
    display(shown.loc[[row[l], row[r]]])
    display(waterfall(model, {c.name: row[f"gamma_{c.name}"] for c in FINAL.model.comparisons}, len(res.records)).
round(2))
```

```
accepted true match with a typo: 6.61 bits → match probability 0.9899 · found by address+address_code+sn_person_name
+person_name_deletions_postal_area
```

CONT_ID	FULL_NAME_KANA	NYS_CNV_P_NAME_KANA	ADDRESS	PHONE_NUM_4	CIF_ID
0000003277	マツモトヒサオ	<NA>	福島県福島市吉倉4-11	0599	5227073594045
0000032891	マツモトヒサウ	<NA>	福島県福島市吉倉4-11	0499	5912064752971

	evidence	level	bits
0	prior	two random records match	-14.98
1	name	fold JW low	3.98
2	company	missing	0.00
3	address	same address	14.01
4	phone	one digit off	4.50
5	cif	else	-0.90

```
rejected non-match closest to the threshold: 2.64 bits → match probability 0.8614 · found by address+address_code
```

CONT_ID	FULL_NAME_KANA	NYS_CNV_P_NAME_KANA	ADDRESS	PHONE_NUM_4	CIF_ID
0000019441	<NA>	セルシスカブシキガイシャ	東京都多摩市連光寺5-18-5	8813	2754477213332
0000020498	ヤマモトカヨ	<NA>	東京都多摩市連光寺5-18-5	6813	4590715084681

	evidence	level	bits
0	prior	two random records match	-14.98
1	name	missing	0.00
2	company	missing	0.00
3	address	same address	14.01
4	phone	one digit off	4.50
5	cif	else	-0.90

Output: train / val / test → artifacts and gold (test is the final report)

Writes accepted pairs, the cumulative clusters and the unresolved records per dataset, plus `medallion/gold/<dataset>.csv` (CONT_ID → cluster). The counter flags configurations scored on test more than once.

In [12]:
```
summary = []
for dataset in DATA.datasets:
    res, records = run(FINAL, dataset), load(DATA, dataset)[0]
    acc = res.scored[res.scored["match_weight"] >= FINAL.min_match_weight].assign(stage=FINAL.stage, rule=FINAL.mo
del.name)
    pairs = acc[[l, r, "stage", "rule", "match_weight", "match_probability", "sources"]].assign(score=acc["match_p
```

```
robability"], kept=True)
    resolved = res.records[[uid, "CLUSTER_ID", "MATCH_STAGE", "MATCH_RULE"]].set_axis([uid, "cluster_id", "resolve
d_stage", "resolved_by"], axis=1)
    unresolved = res.records.loc[res.records["MATCH_STAGE"].isna(), list(records.columns)]
    assert len(unresolved) + resolved["resolved_stage"].notna().sum() == len(records)
    write_stage(DATA, dataset, FINAL.stage, pairs, resolved, unresolved, FINAL)
    write_gold(DATA, dataset, resolved)
    m = res.metrics.iloc[-1]
    append_leaderboard(DATA, "probabilistic:FINAL", FINAL, dataset,
                       {**m.drop("stage").to_dict(), **{f"stage_{x}": v for x, v in res.stage_metrics.items()}})
    summary.append(dict(dataset=dataset, records=len(records), recall_before=res.metrics.iloc[0]["recall"], **m[SH
OW[1:]].to_dict(),
                        accepted=len(acc), seconds=round(res.seconds, 1)))
display(pd.DataFrame(summary).round(4))
display(test_evaluations(DATA))
print("saved", write_final(DATA, "probabilistic_final", FINAL, fit(FINAL, train_records, train_truth, "train"),
                           adopted=ADOPTED, min_match_weight=FINAL.min_match_weight).name)
assert_test_scored_once(DATA)
```

	dataset	records	recall_before	TP	FP	recall	recall_lo	recall_hi	fp_rate_hi	pairs_FP	overmerged	dup_unresolved	accepted	seconds
0	train	30147	0.6923	6207	0	0.8946	0.8872	0.9016	0.0005	0	0	1204	1345	1.6
1	val	10002	0.6933	2042	0	0.9169	0.9047	0.9277	0.0015	0	0	314	474	0.7
2	test	10066	0.7106	2148	0	0.9113	0.8992	0.9221	0.0014	0	0	341	446	0.7

	family	configs_on_test
0	deterministic	1
1	probabilistic	1

```
saved probabilistic_final.json
```

Optuna: component combinations with a cross-validated objective

Each trial switches the scalable generators on or off (the others stay as `FINAL` has them), picks the m method, term frequency, level thresholds and resolver; its acceptance threshold comes from `THRESHOLD_RULE` on its own out-of-fold scores, as `FINAL`'s does. The objectives (**minimise FP**, **maximise recall**) are computed **out-of-fold** on train, so tuning never rewards a model scored on its own fitting data. A trial that would exceed the budget at `budget.target_rows` is pruned. Live dashboard: `http://127.0.0.1:8081`.

In [13]:

```
def spec_for(trial: optuna.Trial | optuna.trial.FixedTrial) -> ProbSpec:
    on = {b for b in PTUNE.toggle_blockers if trial.suggest_categorical(f"use:{b}", [True, False])}
    fixed = {b.name for b in FINAL.blockers} - set(PTUNE.toggle_blockers)  # the other generators stay as FINAL ha
s them
    m_method = trial.suggest_categorical("m_method", list(PTUNE.m_methods))
    tf = {c.name: c.tf for c in SPEC.model.comparisons} if trial.suggest_categorical("term_frequency", list(PTUNE.
tf_options)) else {}
    model = replace(FINAL.model, m_method=m_method,  # tf keys come from SPEC: FINAL may have dropped them
                    comparisons=tuple(replace(c, tf=tf.get(c.name)) for c in FINAL.model.comparisons))
    for (comparison, label), (lo, hi, step) in PTUNE.level_thresholds.items():
        model = with_level(model, comparison, label, threshold=trial.suggest_float(f"{comparison}|{label}", lo, h
i, step=step))
    return replace(FINAL, blockers=tuple(b for name, b in BY_NAME.items() if name in fixed | on), model=model,
                   resolver=PTUNE.resolvers[trial.suggest_categorical("resolver", list(PTUNE.resolvers))])


def objective(trial: optuna.Trial) -> tuple[float, float]:
    spec = spec_for(trial)
    if not GROWTH.loc[[b.name for b in spec.blockers], "within_budget"].all():
        raise optuna.TrialPruned("over budget at target_rows")
    try:
        weight = threshold_of(spec)
    except OverBudget as e:
        raise optuna.TrialPruned(str(e))
    at = cv_threshold_curve(cv(spec), spec, train_prev_pairs, train_truth, FOLDS, DB, DATA, [weight]).iloc[0]
    for x, v in dict(min_match_weight=weight, recall_lo=at["recall_lo"], recall_hi=at["recall_hi"],
                     candidates=summed(cv(spec))["candidates"]).items():
        trial.set_user_attr(x, plain(v))
    return float(at["FP"]), float(at["recall"])


storage = start_dashboard(PTUNE)
print(f"study: O(T · k · (fit + candidates + score + resolve)), T = {PTUNE.n_trials}, k = {CV_FOLDS}; "
      f"each trial re-samples u only for the comparisons it changed (u is cached per comparison and fold)")
study = new_study(PTUNE, storage, ["FP", "recall"], ["minimize", "maximize"])
study.optimize(objective, n_trials=PTUNE.n_trials, show_progress_bar=True)
```

```
Optuna dashboard: http://127.0.0.1:8081  (storage sqlite:////Users/marvinma/projects/entity matching/data/artifacts/optuna/probabilistic_matching.db)
study: O(T · k · (fit + candidates + score + resolve)), T = 30, k = 3; each trial re-samples u only for the comparisons it changed (u is cached per comparison and fold)
study 'probabilistic_20261105_084412' → watch it live in the dashboard
```

```
Bottle v0.13.4 server starting up (using WSGIRefServer())...
Listening on http://127.0.0.1:8081/
Hit Ctrl-C to quit.
```

Pareto front, re-scored on val

The suggested point is the cheapest configuration within the noise of the best 0-FP val recall (`pick_within_noise` ; none if no point has 0 FP on val). It is judged by the same rule as `DECISIONS` ("better"), against `FINAL` ; to adopt a passing point, add it as a `CHANGES` + `DECISIONS` entry.

In [14]:

```
front, seen = [], set()
for t in sorted(study.best_trials, key=lambda t: (t.values[0], -t.values[1])):
    spec = replace(spec_for(optuna.trial.FixedTrial(t.params)), min_match_weight=t.user_attrs["min_match_weight"])
    if repr(spec) in seen:
        continue
    seen.add(repr(spec))
    v = run(spec, "val").metrics.iloc[-1]
    front.append(dict(trial=t.number, cv_FP=t.values[0], cv_recall=t.values[1], val_FP=v["FP"], val_recall=v["recall"],
                      val_recall_lo=v["recall_lo"], val_recall_hi=v["recall_hi"], val_zero_fp=v["FP"] == 0,
                      candidates=t.user_attrs["candidates"], min_match_weight=t.user_attrs["min_match_weight"],
                      term_frequency=t.params["term_frequency"], m_method=t.params["m_method"], spec=spec))
front = pd.DataFrame(front)
display(front.drop(columns="spec").drop_duplicates(["val_FP", "val_recall", "candidates"]).round(4))  # one row per distinct outcome
best = pick_within_noise(front, "val_recall", "val_recall_lo", "val_recall_hi", "val_zero_fp", "candidates") if len(front) else None
if best is None:
    print("no Pareto point has 0 FP on val: FINAL stays")
else:
    spec = front.at[best, "spec"]
    on_cv = paired_test(joined(cv(FINAL)), joined(cv(spec)), train_true)
    on_val = paired_test(joined(run(FINAL, "val")), joined(run(spec, "val")), val_true)
```

```
    print(f"suggested: trial {front.at[best, 'trial']} → adopt by rule ('better' vs FINAL): {adopt('better', on_cv, on_val, DECISION_P)}")
    print(f"cross-validated train {on_cv} · val {on_val}")
```

	trial	cv_FP	cv_recall	val_FP	val_recall	val_recall_lo	val_recall_hi	val_zero_fp	candidates	min_match_weight	term_frequency	m_method
0	3	0.0	0.9075	0	0.9142	0.9019	0.9252	True	88466	6.0	True	em

```
suggested: trial 3 → adopt by rule ('better' vs FINAL): False
cross-validated train {'TP_gains': 0, 'TP_losses': 12, 'p': np.float64(0.00048828125), 'FP_added': 0, 'FP_removed': 0} · val {'TP_gains': 0, 'TP_losses': 6, 'p': np.float64(0.03125), 'FP_added': 0, 'FP_removed': 0}
```